High Performance Computing

MapReduce Assignment

Environment Setup

In this exercise, we will simulate the main stages of MapReduce using Python and standard command-line tools. A Hadoop cluster is not required.

You need:

- Python 3;
- a Linux, macOS, or WSL terminal;
- the standard sort command.

Linux / Ubuntu / WSL

Check that Python is installed:

```
python3 --version
```

If Python is not installed:

```
sudo apt update
sudo apt install python3
```

Windows users are recommended to use the Ubuntu/WSL environment that was also used for the MPI exercise.

macOS

Check Python using:

```
python3 --version
```

If necessary, Python can be installed using Homebrew:

```
brew install python
```

Objectives

The purpose of this exercise is to understand the basic MapReduce programming model and its data flow. You will implement mapper and reducer programs, observe how intermediate key–value pairs are grouped during the shuffle and sort stage, perform word counting, investigate the role of a combiner and partitioner, and implement a simple inverted index. You will also relate the practical exercises to scheduling, data locality, fault tolerance, and distributed storage concepts discussed in the lecture.

Part 1: Understanding Map and Reduce

Consider the following two documents:

doc1: data science data
doc2: science needs data

For the Word Count problem, the mapper produces:

$$(\text{word}, 1)$$

for every occurrence of a word.

1. Mapper Output

Write all intermediate key–value pairs produced by the mapper.
For example:

$$(\text{data}, 1)$$

Complete the table.

Document	Mapper Output
doc1	(data, 1), (science, 1), (data, 1)
doc2	(science, 1), (needs, 1), (data, 1)

Document	Mapper Output
doc1	
doc2	

2. Shuffle and Sort

Group all intermediate values having the same key.
Complete:

$$\text{data} \rightarrow [1, 1, 1 \quad]$$

$$\text{science} \rightarrow [1, 1 \quad]$$

$$\text{needs} \rightarrow [1 \quad]$$

3. Reduce

The reducer calculates:

$$\text{count}(w) = \sum_i v_i.$$

Write the final output:

$$(\text{data}, 3 \quad)$$

$$(\text{science}, 2 \quad)$$

$$(\text{needs}, 1 \quad)$$

Answer:

1. What is the input key to the mapper in a document-processing MapReduce application?

Document ID, for example doc1 or doc2. In Hadoop, for a text file, it is the position of the line in the file.

2. What is the input value?

The text of the document, for example “data science data”

3. What key is produced by the Word Count mapper?

The word

4. What does the shuffle and sort stage do?

Collects the pairs from all mappers, puts pairs with the same key into one group and sorts the groups by key. Each group goes to its reducer

5. Why must all intermediate values having the same key reach the same reducer?

The reducer adds up all the 1s for one word. If the pairs for data went to two reducers, we would get (data, 2) and (data, 1) instead of the correct (data, 3)

Part 2: Implementing Word Count

Create a text file called:

input.txt

containing at least five lines of text.
For example:

```
Parallel computing can process large datasets
MapReduce processes data using map and reduce
Parallel systems can process data faster
Data can be distributed across many machines
MapReduce is designed for distributed processing
```

Mapper

Create:

mapper.py

The mapper should:

1. read input from standard input;
2. convert words to lowercase;
3. extract individual words;
4. output one pair

(word, 1)

for every word.

Use tab-separated output:

word 1

You may start with:

```
import sys
import re

for line in sys.stdin:

    line = line.strip().lower()

    words = re.findall(r"[a-z0-9']+", line)

    for word in words:

        # Output the intermediate key-value pair
        # TODO
```

Run the mapper:

```
python3 mapper.py < input.txt
```

Save the output:

```
python3 mapper.py < input.txt > map_output.txt
```

Inspect:

```
cat map_output.txt
```

Shuffle and Sort

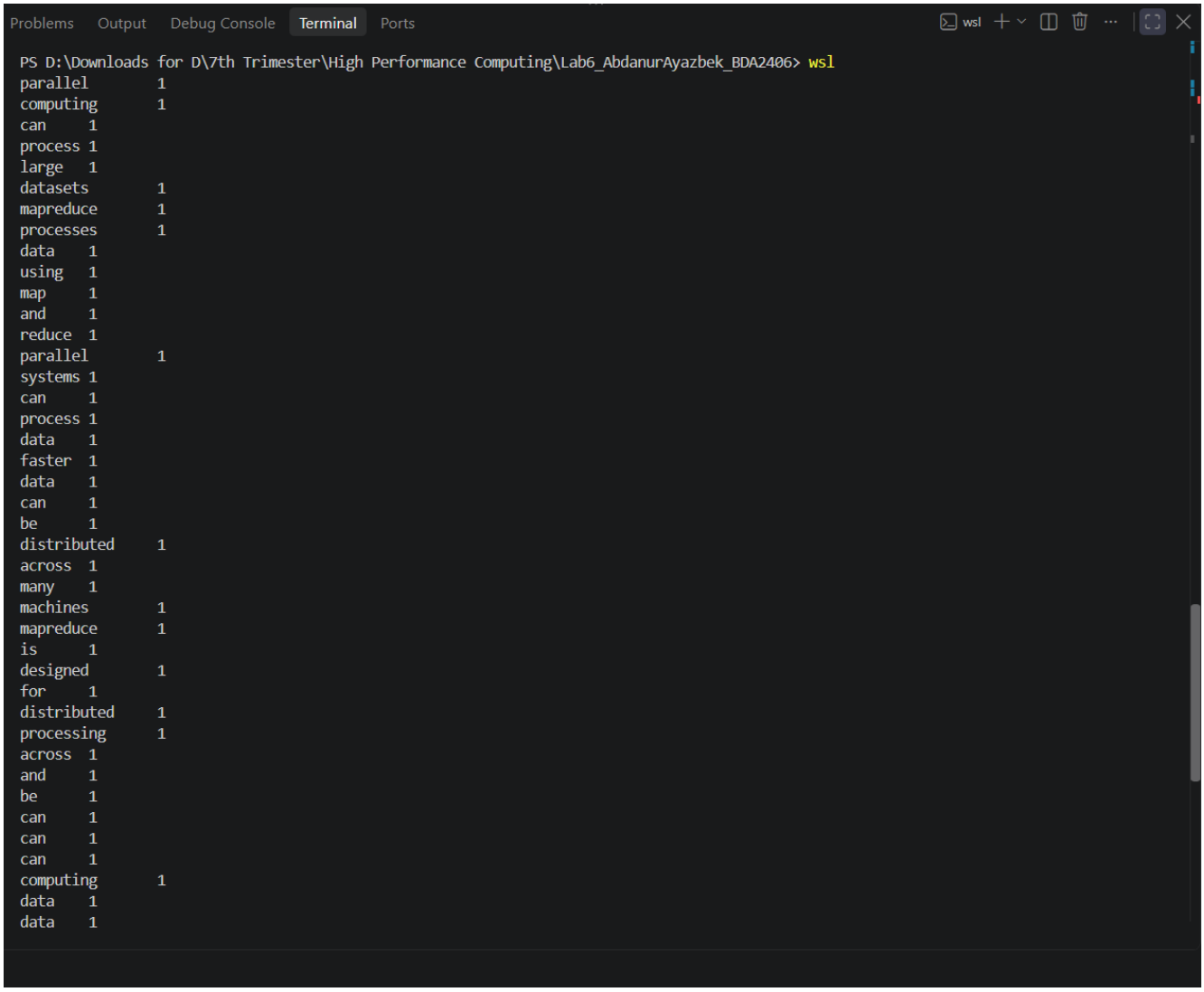
MapReduce groups intermediate data according to the key.

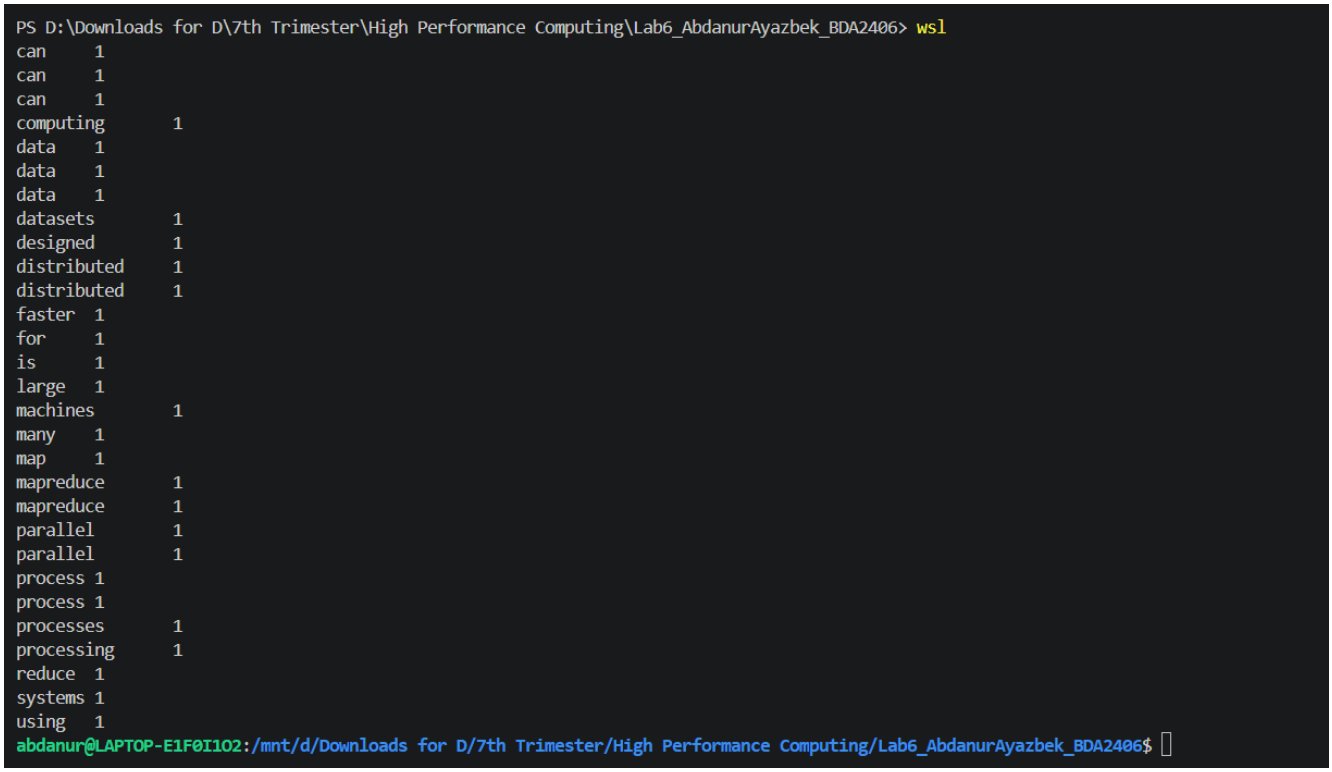
For this exercise, simulate the shuffle and sort stage using:

```
sort
```

Run:

```
python3 mapper.py < input.txt | sort
```


Problems Output Debug Console Terminal Ports
wsl
PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab6_AbdanurAyazbek_BDA2406> wsl
parallel 1
computing 1
can 1
process 1
large 1
datasets 1
mapreduce 1
processes 1
data 1
using 1
map 1
and 1
reduce 1
parallel 1
systems 1
can 1
process 1
data 1
faster 1
data 1
can 1
be 1
distributed 1
across 1
many 1
machines 1
mapreduce 1
is 1
designed 1
for 1
distributed 1
processing 1
across 1
and 1
be 1
can 1
can 1
can 1
computing 1
data 1
data 1


PS D:\Downloads for D\7th Trimester\High Performance Computing\Lab6_AbdanurAyazbek_BDA2406> wsl
can 1
can 1
can 1
computing 1
data 1
data 1
data 1
datasets 1
designed 1
distributed 1
distributed 1
faster 1
for 1
is 1
large 1
machines 1
many 1
map 1
mapreduce 1
mapreduce 1
parallel 1
parallel 1
process 1
process 1
processes 1
processing 1
reduce 1
systems 1
using 1
abdanur@LAPTOP-E1F0I1O2:/mnt/d/Downloads for D/7th Trimester/High Performance Computing/Lab6_AbdanurAyazbek_BDA2406$

Observe how equal keys become adjacent.
Answer:

1. Does sort calculate the word counts?

No. It only puts equal words next to each other

2. What useful property does sorting provide to the reducer?

Equal keys come one after another. So the reducer only compares each word with the previous one and keeps just one word and one count in memory

Reducer

Create:

reducer.py

The reducer receives sorted intermediate pairs and calculates the total count for every word.
You may start with:

```python
import sys

current_word = None
current_count = 0

for line in sys.stdin:

    line = line.strip()

    word, count = line.split("\t")
    count = int(count)

    if word == current_word:

        # TODO: update count
        pass

    else:

        # TODO: output previous word if necessary

        # TODO: start counting a new word
        pass

# TODO: output the final word
```

Run the complete MapReduce pipeline:

```
python3 mapper.py < input.txt | sort | python3 reducer.py
```

Save the final result:

```
python3 mapper.py < input.txt | sort | \
python3 reducer.py > output.txt
```

```
across  1
and     1
be      1
can     3
computing       1
data    3
datasets        1
designed        1
distributed     2
faster  1
for     1
is      1
large   1
machines        1
many    1
map     1
mapreduce       2
parallel        2
process 2
processes       1
processing      1
reduce  1
systems 1
using   1
abdanur@LAPTOP-E1F0I1O2:/mnt/d/Downloads for D/7th Trimester/High Performance Computing/Lab6_AbdanurAyazbek_BDA2406$
```

Part 3: Follow the Data Flow

Choose the following input:

red blue red
blue green
red green blue

Write the data after each stage.

```
red     1
blue    1
red     1
blue    1
green   1
red     1
green   1
blue    1
blue    1
blue    1
blue    1
green   1
green   1
red     1
red     1
red     1
blue    3
green   2
red     3
abdanur@LAPTOP-E1F0I1O2:/mnt/d/Downloads for D/7th Trimester/High Performance Computing/Lab6_AbdanurAyazbek_BDA2406$
```

Map Output

red 1
blue 1
red 1
blue 1
green 1
red 1
green 1
blue 1

After Shuffle and Sort

blue 1
blue 1
blue 1
green 1
green 1
red 1
red 1
red 1

Final Reduce Output

blue 3
green 2
red 3

Answer:

1. Does a mapper need to know the final number of occurrences of a word?

No. It only outputs (word, 1) for every word it sees. The total is calculated later by the reducer

2. Does one mapper need the output of another mapper before it can process its own input?

No. Each mapper works on its own part of the input, so all mappers can run at the same time

3. Which stage brings together values generated by different mappers?

Shuffle and sort

4. Is the reducer executed before all relevant mapper outputs become available?

No. The reducer starts only after all mapper outputs for its keys are ready. Otherwise the count would be incomplete

Part 4: Combiner

A combiner performs local aggregation after mapping and before the global shuffle.

Suppose two mappers independently process:

Mapper 1 input:
data data science data

Mapper 2 input:
science data science

Without a combiner, Mapper 1 produces:

data 1
data 1
science 1
data 1

Instead, a combiner could locally produce:

data 3
science 1

Task

Create:

combiner.py

You may reuse the same aggregation logic as the reducer.
Create two input files:

part1.txt
part2.txt

Run:

```
python3 mapper.py < part1.txt | sort | \
python3 combiner.py > combined1.txt

python3 mapper.py < part2.txt | sort | \
python3 combiner.py > combined2.txt
```

Combine both local outputs:

```
cat combined1.txt combined2.txt | sort | \
python3 reducer.py
```

Compare the number of records before and after using the combiner:

```
wc -l map_output.txt
wc -l combined1.txt
```

```
data    3
science 1
data    1
science 2
data    4
science 3
32 map_output.txt
2 combined1.txt
4
abdanur@LAPTOP-E1F0I1O2:/mnt/d/Downloads for D/7th Trimester/High Performance Computing/Lab6_AbdanurAyazbek_BDA2406$
```

Answer:

1. Does a combiner change the final Word Count result?

No. The final result is data 4, science 3, the same as without a combiner

2. Does it reduce the number of intermediate records that may need to be transferred?

Yes. For part1.txt the mapper produced 4 records, and the combiner reduced them to 2

3. Why can reducing intermediate data reduce network traffic?

Intermediate data is sent over the network from mappers to reducers. Fewer records means less data to send, so the transfer takes less time

4. Why can a summation operation safely use a combiner?

Addition gives the same result in any order and grouping: (1 + 1 + 1) + 1 = 1 + 1 + 1 + 1. So adding partial sums first does not change the total

Part 5: Partitioning

Assume that the system has two reducers:

$$R_0, \qquad R_1.$$

The partitioner determines which reducer receives each intermediate key.
For this exercise, use the simple rule:

$$\text{partition}(k) = \left(\sum_{c \in k} \text{ASCII}(c)\right) \bmod 2.$$

For example, all occurrences of the same word must produce the same partition number.

```
data 410 0 R0
science 730 0 R0
parallel 845 1 R1
computer 879 1 R1
hadoop 635 1 R1
reduce 632 0 R0
abdanur@LAPTOP-E1F0I1O2:/mnt/d/Downloads for D/7th Trimester/High Performance Computing/Lab6_AbdanurAyazbek_BDA2406$
```

Calculate the partition for:

Key	ASCII sum	Reducer
data	410	R0
science	730	R0
parallel	845	R1
computer	879	R1
hadoop	635	R1
reduce	632	R0

Answer:

1. Can the same intermediate key be sent to two different reducers in a normal per-key reduction?

No. The partition function always gives the same number for the same key, so all its values go to one reducer

2. Can two different keys be sent to the same reducer?

Yes. For example, data, science and reduce all go to R0

3. Why is partitioning necessary when several reducers are used?

Each reducer must know which keys it is responsible for. The partitioner makes sure every key goes to exactly one reducer, so each word is counted completely in one place

4. Does partitioning guarantee that all reducers receive exactly the same amount of work?

No. It only splits keys, not records. One reducer can get keys that appear very often, for example data, and receive much more work than the other

Part 6: Inverted Index

An inverted index maps every word to the documents containing that word.
Create the file:

documents.txt

containing:

```
doc1    parallel computing uses many processors
doc2    mapreduce processes large data
doc3    parallel data processing is useful
doc4    computing systems process data
```

Each line contains:

(document ID, text).

Create:

index_mapper.py

The mapper should emit:

(word, document ID)

for every word.
For example:

```
parallel    doc1
computing   doc1
```

Then create:

index_reducer.py

The reducer should produce output similar to:

computing doc1,doc4
data doc2,doc3,doc4
parallel doc1,doc3

Do not print the same document ID more than once for the same word.
Run:

```
python3 index_mapper.py < documents.txt | sort | \
python3 index_reducer.py
```

```
parallel        doc1
computing       doc1
uses    doc1
many    doc1
processors      doc1
mapreduce       doc2
processes       doc2
large   doc2
data    doc2
parallel        doc3
data    doc3
processing      doc3
is      doc3
useful  doc3
computing       doc4
systems doc4
process doc4
data    doc4
computing       doc1,doc4
data    doc2,doc3,doc4
is      doc3
large   doc2
many    doc1
mapreduce       doc2
parallel        doc1,doc3
process doc4
processes       doc2
processing      doc3
processors      doc1
systems doc4
useful  doc3
uses    doc1
abdanur@LAPTOP-E1F0I1O2:/mnt/d/Downloads for D/7th Trimester/High Performance Computing/Lab6_AbdanurAyazbek_BDA2406$
```

Questions

1. What is the intermediate key in this application?

The word

2. What is the intermediate value?

The document ID, for example doc1

3. Why must all occurrences of one word be sent to the same reducer?

The reducer builds the full list of documents for one word. If one word went to two reducers, each would give only part of the list, for example data doc2 on one reducer and data doc3,doc4 on the other

4. How is this application different from Word Count?

In Word Count the value is 1 and the reducer adds numbers. Here the value is a document ID, and the reducer collects a list of unique IDs instead of a sum

Part 7: MapReduce Runtime

For each operation below, state whether it is mainly the responsibility of the **programmer** or the **MapReduce runtime**.

Operation	Programmer / Runtime
Define the mapper	Programmer
Define the reducer	Programmer
Schedule map tasks	Runtime
Group intermediate values by key	Runtime
Shuffle intermediate data	Runtime
Detect failed workers	Runtime
Restart failed tasks	Runtime
Choose what the application computes	Programmer

Answer:

1. Why is hiding scheduling and communication details useful to the programmer?

The programmer only writes the mapper and reducer and does not have to deal with many machines, network transfers and failures. This makes programs shorter, easier to write and with fewer bugs

2. What happens between the map and reduce phases?

Shuffle and sort. The runtime collects the mapper outputs, sends each key to its reducer through the network, groups the values by key and sorts them

3. Why can intermediate data movement become expensive?

Mappers and reducers usually run on different machines, so intermediate data has to go over the network, which is much slower than reading from local memory or disk. With big input, the mappers can produce a lot of intermediate data

Part 8: Data Locality and Distributed Storage

Answer the following questions briefly.

1. Why does a MapReduce scheduler try to execute a map task close to the physical location of its input data?

Moving a large input block over the network is slow. If the task runs on the machine that already stores the block, it reads the data from its local disk, and the network stays free

2. What is meant by “moving computation to the data”?

Instead of sending big data to the program, the system sends the small program to the machines where the data is stored

3. What is the purpose of a distributed file system in a MapReduce environment?

It splits big files into blocks and stores them on many machines. Every machine can then read its own part of the data at the same time, and the data survives if one machine fails

4. What is the role of the HDFS NameNode?

Stores the metadata: the list of files, which blocks each file has and on which DataNodes each block is stored. Clients ask the NameNode where the blocks are

5. What is the role of an HDFS DataNode?

Stores the actual file blocks on its local disk and sends them to clients when they read the file

6. Does the NameNode normally transfer the actual file contents between the client and DataNodes?

No. The NameNode only tells the client where the blocks are. The client then reads and writes the data directly with the DataNodes

7. Why are file blocks replicated on several machines?

Machines fail often in a big cluster. With copies on several machines, a block is not lost when one machine breaks

8. How can replication improve fault tolerance?

If a DataNode fails, the data is read from another copy and tasks continue. The NameNode then makes a new copy on another machine to bring back the normal number of copies

Part 9: Short Discussion

Answer each question in 2–4 sentences.

1. What is the difference between the Map phase and the Reduce phase?

The Map phase processes each part of the input independently and turns it into key value pairs, for example (data, 1). The Reduce phase takes all values of one key and combines them into the final result, for example (data, 3). Map works on raw input, Reduce works on grouped intermediate data

2. What is the purpose of the shuffle and sort stage?

It brings all values with the same key from all mappers to one reducer and sorts them by key. Without it, the values of one word would stay spread across many machines, and the reducer could not produce a complete result

3. Why are key–value pairs central to the MapReduce model?

The key decides where each piece of data goes: all pairs with the same key end up in the same reducer. This lets the system split the work across many machines without the programmer writing any communication code

4. What is the purpose of a combiner?

A combiner partly reduces the mapper output on the same machine before it is sent over the network. For example, it turns (data, 1) three times into (data, 3). This means less data to transfer, and the final result stays the same

5. What is the difference between a combiner and a partitioner?

A combiner shrinks the data: it merges values of the same key locally after a mapper. A partitioner does not change the data, it only decides which reducer gets each key. So the combiner reduces how much data is sent, and the partitioner decides where it is sent

6. Why is Word Count naturally parallelizable?

Every word can be counted in every part of the text independently, so mappers never need each other's results. The partial counts are then simply added up, and addition gives the same result in any order

7. Give one example of a problem that is well suited to MapReduce.

Counting how many times each page was visited, using huge web server logs. Each log line can be processed independently, and the counts are added up by page URL, exactly like Word Count

8. Give one example of a problem that would be difficult to express efficiently as a single

MapReduce job.

K-means clustering. It repeats the same steps many times until the result stops changing, and every round needs the result of the previous one. A single MapReduce job can do only one round, so the algorithm needs many jobs one after another, each reading and writing all the data again

Submission

Submit:

1. mapper.py;
2. reducer.py;
3. combiner.py;
4. index_mapper.py;
5. index_reducer.py;
6. input.txt and documents.txt;
7. the final Word Count output;
8. the final inverted-index output;
9. completed tables;
10. answers to all questions.

All submitted Python programs should run using Python 3.